

DSCI 4412 Term Project Proposal

April 26, 2026

Team Members

Name	ID
Omar Moustafa	900222400
Nour Kahky	900221042
Nour Moghazi	900225966
Abdelrahman Bayoumy	900223218

Chosen Topic:

“The Catalyst Optimizer Under the Microscope”

Why Does This Topic Interest Us?

Throughout Chapters 3, 7, and 8, we observed that query writing style can dramatically improve and impact performance. The natural next step is to learn and understand why these changes in performance take place. We want to know how and when certain optimizations fail and can be defeated. The promise of the Catalyst optimizer is that various equivalent queries should end up with the same optimized plan as each other, so we aim to test these particular queries and measure where and when the optimized plans diverge across syntactically different but logically identical queries and tasks.

Experimental Design

Experiment #1: Equivalent Queries for Total Revenue per Region

Research Question: Write four equivalent queries for “total revenue per region” using DataFrame API, SQL, RDD, and a mixed approach. Compare physical plans. Are they identical? If not, why?

What we will vary: Query style: DataFrame API, Spark SQL, RDD transformation, and mixed SQL/DataFrame approach.

What we will measure: Optimized plan, physical plan, execution time, shuffle operations, and whether Spark uses the same aggregation strategy.

Tables and columns:

- **orders:** order_id, customer_id, total
- **customers:** customer_id, region

Experiment #2: Predicate Pushdown Defeat

Research Question: Can predicate pushdown be defeated using a Python UDF, a non-deterministic function (`rand()`), and a complex CASE expression? Which does Catalyst push down?

What we will vary: Filter expression type: simple deterministic filter, Python UDF filter, non-deterministic `rand()` filter, and complex CASE WHEN filter.

What we will measure: Whether filters appear under PushedFilters, optimized plan changes, physical plan changes, and execution time.

Tables and columns:

- **orders:** `order_id`, `total`, `subtotal`, `tax`, `shipping`, `order_date`

Experiment #3: Join Reordering

Research Question: Write `customers` $\bowtie$ `orders` $\bowtie$ `order_items` in all 6 orderings. Compare the optimized plans. Does Spark find the same plan every time?

What we will vary: Join order across the three tables.

What we will measure: Optimized logical plan, physical join order, join strategy, shuffle/exchange operations, and execution time.

Tables and columns:

- **customers:** `customer_id`, `region`
- **orders:** `order_id`, `customer_id`
- **order_items:** `order_id`, `product_id`, `quantity`, `unit_price`

Experiment #4: Extended Plan Annotation

Research Question: Run `explain("extended")` on a complex query and annotate each rewrite between parsed, analyzed, optimized, and physical plans. Which specific optimization rules fired?

What we will vary: Query complexity by adding joins, filters, computed columns, aggregation, and sorting.

What we will measure: Which parts of the query are rewritten, removed, pushed down, combined, or rearranged by Catalyst.

Tables and columns:

- **customers:** `customer_id`, `region`, optional customer attributes
- **orders:** `order_id`, `customer_id`, `total`, `order_date`
- **order_items:** `order_id`, `quantity`, `unit_price`

Experiment #5: Expression Equivalence Recognition

Research Question: Does Catalyst recognize that `orders.total = orders.subtotal + orders.tax + orders.shipping`?

What we will vary: Filter condition: filtering directly on `total` versus filtering on `subtotal + tax + shipping`.

What we will measure: Optimized plan similarity, physical plan similarity, pushed filters, execution time, and result equivalence.

Tables and columns:

- **orders:** `total`, `subtotal`, `tax`, `shipping`

★ Experiment #6: Disabling Optimization Rules

Research Question: Disable optimization rules using `spark.sql.optimizer.excludedRules`. Which rule's absence hurts performance the most?

What we will vary: Catalyst optimization rules disabled one at a time.

What we will measure: Runtime, optimized plan changes, physical plan changes, shuffle operations, pushed filters, and join strategy changes.

Tables and columns:

- `customers`, `orders`, and `order_items`
- Columns used in the complex query from Experiment 4.

Experimental Methodology

For each question, we will run the same six-step structure on the seeded orders, customers, and order_items tables:

1. Write the query variants.
2. Run `explain("extended")`.
3. Record the parsed, analyzed, optimized, and physical plans.
4. Measure runtime using repeated actions.
5. Inspect partitions/shuffles where relevant.
6. Compare whether Catalyst produced the same or different optimized/physical plans.

Variables we will control: We will use the same Spark session, same dataset version, same input tables, same cluster/local settings, same number of timing runs, and the same action trigger for all query variants. We will also control cache state by clearing cache before timing unless the experiment specifically involves caching.

Variables we will vary: We will change the query-writing style, filter expression type, join ordering, logical query complexity, equivalent arithmetic expression, and optimizer rule settings. These variations allow us to isolate how Catalyst responds to different query forms while keeping the underlying data and intended result constant.

Metrics: We will measure execution time, optimized logical plan differences, physical plan differences, number of stages/tasks where applicable, shuffle operations, pushed filters, join strategy, and whether the final query result remains equivalent across query versions.

Scope Check

Minimum Viable Project: Conduct five experiments answering Questions 1 to 5 to exceed the minimum and meet the target number of questions and experiments for a four-person group.

Stretch Goal: Conduct an additional sixth experiment answering Question 6, which is an ambitious question that works to analyze how performance degrades from disabling certain optimization rules, to meet the strong criterion for the work done by a four-person group.

Division of Work

Research Question / Project Task	Led By
Question #1 (DataFrame API & SQL Equivalence)	Nour Moghazi
Question #2 (Defeating Predicate Pushdown)	Nour Kahky
Question #3 (Join Reordering Consistency)	Abdelrahman Bayoumy
Question #4 (Extended Explanation of Complex Queries)	Omar Moustafa
Question #5 (Algebraic Equivalence)	Abdelrahman Bayoumy & Nour Moghazi
Question #6 (Disabling Optimization Rules)	Omar Moustafa & Nour Kahky
Writing the report	All team members
Designing the powerpoint presentation	All team members